Operating Systems Course: Lab02

Laboratory number 02

Exercise 01: Analysis of concurrent programs

Represent the *control flow graph* (CFG) and the *process generation tree* of the following pieces of code. In addition, indicate the output produced on the video. Check the results you predicted, by executing the program.

A.

lab02_e01_A.c

```
for (i=1; i<=2; i++) {
  if (!fork ())
    printf ("%d\n", i);
}
printf ("%d\n", i);
```

B.

lab02_e01_B.c

```
for(i=3; i>1; i--) {
  if (fork ())
    printf ("%d\n", i);
}
printf ("%d\n", i);
```

C.

lab02_e01_C.c

```
for (i=0; i<2; i++) {
  if (fork ())
    fork ();
}
printf ("%d\n", i);
```

D.

lab02_e01_D.c

```
for (i=2; i>=1; i--) {
  if (!fork ())
    printf ("%d\n", -i);
  else
    printf ("%d\n", i);
}
```

Exercise 02: System call fork and shell commands related to processes

A program receives two integer values on the command line, called `n` and `t`. The program (parent process) must pr 2

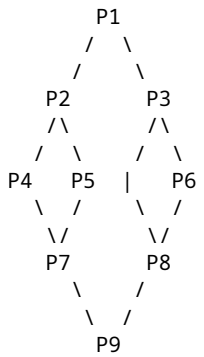
children and terminate. In turn, each child must produce 2 children and terminate. This sequence of operations must continue until `2 ^ n` processes on the leaves of the tree are produced/running. The leaf processes wait for `t` seconds and display (on screen) a termination message. Note that each process (in the tree) produces two other processes. Only those on the leaves of the tree sleep and display a message. What is the order of termination of the processes? Is it always the same? How can they be recognized (ppid)?

Once the program has been executed:

- look for the pid of the various processes with the command ps
    1. from the same shell
    2. from another shell
- terminate the execution of some processes with the kill command executed
    1. from the same shel
    2. from another shell

Exercise 03: Precedence graph

Using the system calls fork and wait, realize the following precedence graph.



Note that all arcs are oriented downwards. Each Pi process corresponds to a print message (print the string “Pi” and the PID of the process).

Check the precedence are respected by inserting the system call `sleep` in the various branches of the program.

If you found any error, or if you want to partecipate to the editing of this wiki, please contact: admin [at] skenz.it

You can reuse, distribute or modify the content of this page, but you must cite in any document (or webpage) this url: https://www.skenz.it/os/lab02